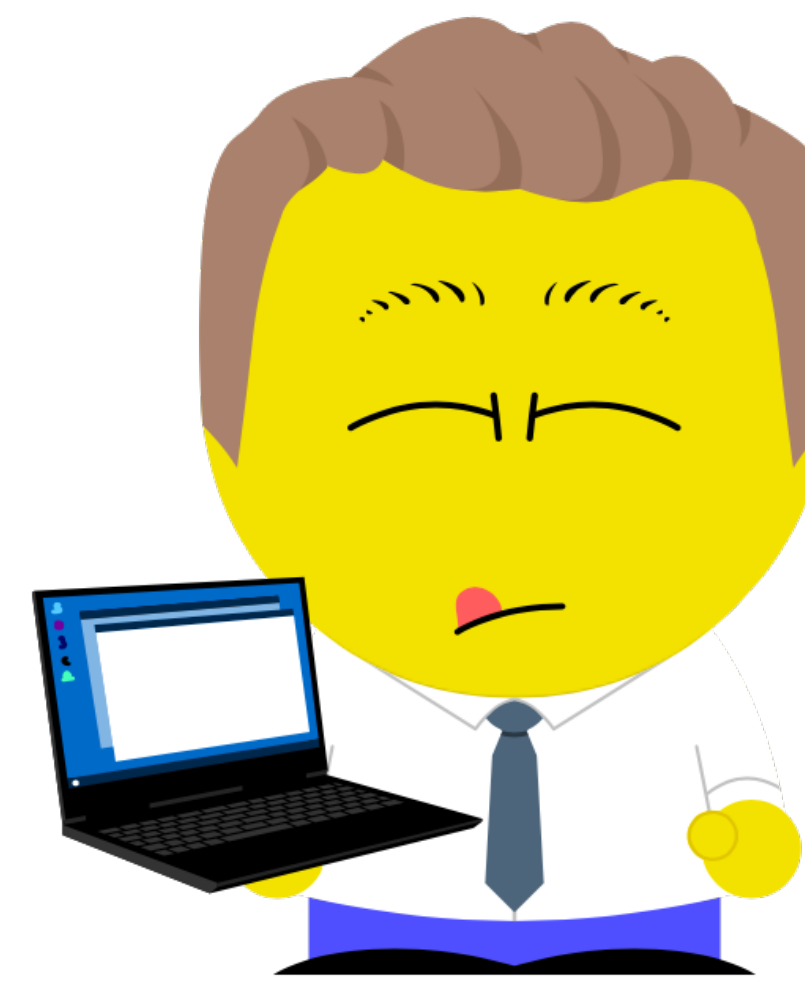




IN-CLASS EXERCISES



GARBAGE COLLECTION 1

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`gc-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip gc-1-exercises.zip
```

```
> cd gc-1-exercises
```

```
> ls
```

Task 1

- Explain these 3 memory management issues:

- 1.Heap fragmentation

- 2.Memory leaks

- 3.Dangling references

Task 2

- Explain these 3 issues that appear when the GC finds the object graph:
 - Find the roots
 - Find internal pointers
 - Find the beginning of objects

Task 3

- Explain these 2 issues that appear when the GC compacts the heap:
 - Move the live objects
 - Update pointers to live objects

Task 4

- Describe the reference counting GC Algorithm.
- What issues occur with this algorithm?

Task 5

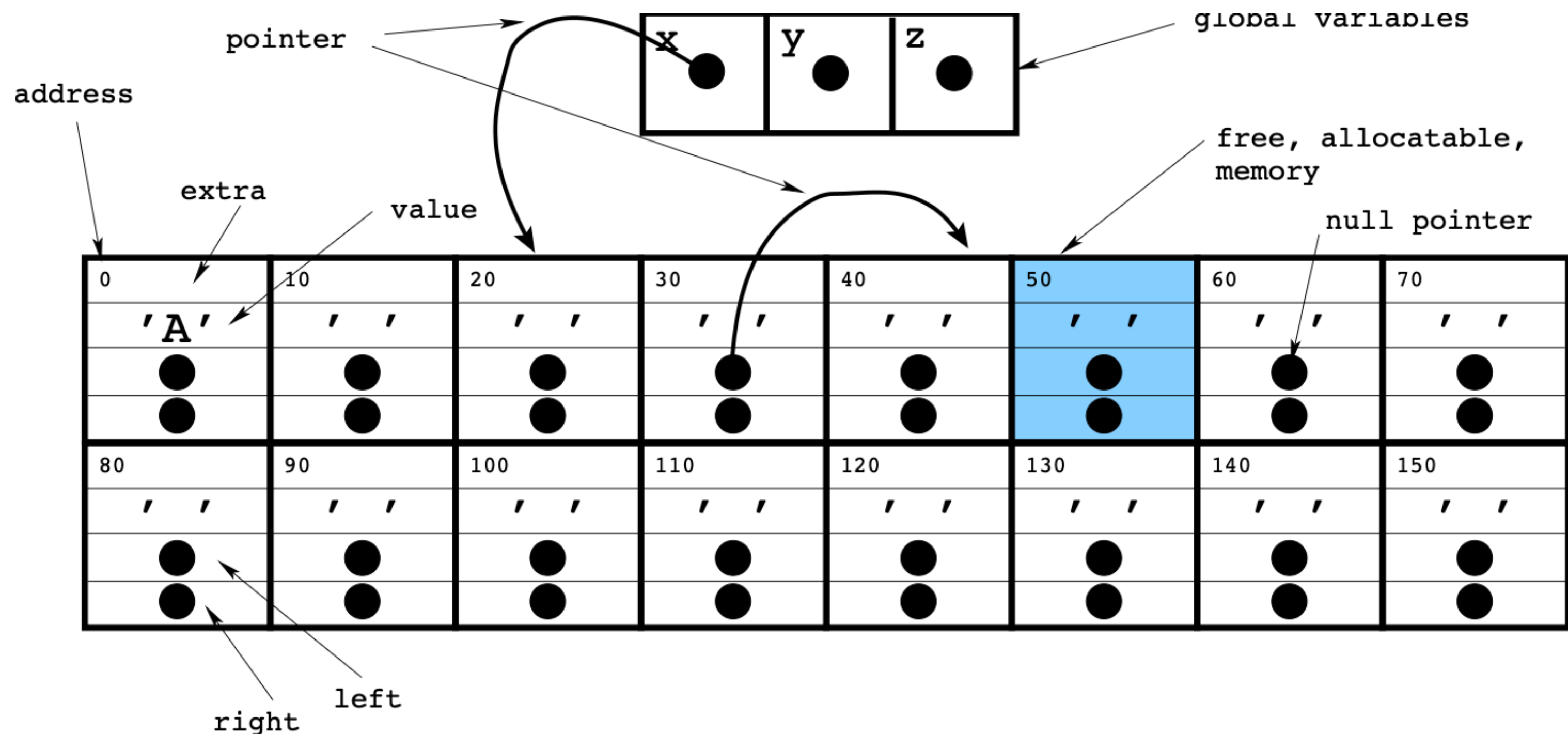
- How does a GC algorithm find internal pointers in a heap object?

Task 5

- Describe these 3 GC strategies:
 - stop-the-world garbage collection
 - incremental garbage collection
 - and concurrent garbage collection
- What are the advantages and disadvantages of each strategy?

Task 6(a)

- Consider the Java program Cons on the next page. This program allocates memory on a heap of size 160 bytes, where each Cons node takes up 10 bytes, which we illustrate like this:



- Here, Extra refers to any additional information that a garbage collection algorithm might want to store in each node.

```
class Cons {
    public Cons left;
    public Cons right;
    public char value;

    public Cons(
        char value,
        Cons left,
        Cons right) {
        this.value = value;
        this.left = left;
        this.right = right;
    }

    static Cons x, y, z;
```

```
public static void main(String[] args) {
    x = new Cons('A', null, null);
    Cons b = new Cons('B', x, null);
    x.left = b;

    Cons f = new Cons('F', null, null);
    Cons e = new Cons('E', f, null);
    Cons d = new Cons('D', null, e);
    z = new Cons('C', d, e );

    b = d = f = e = null;

    <<BEFORE>>

    x = null;
    z.right.left = null;

    <<AFTER>>

    }
}
```

1. What does the heap look like at the point <<**BEFORE**>>, assuming that no garbage collection has occurred?
Assume that nodes are allocated on the heap in the order that the new:s occur in the program.



0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

2. What does the heap look like at the point <<AFTER>>, assuming that a **reference counting garbage collection** occurred? Mark all memory that the program is now able to allocate by shading the free nodes.



0	10	20	30	40	50	60	70
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●
80	90	100	110	120	130	140	150
/	/	/	/	/	/	/	/
●	●	●	●	●	●	●	●
●	●	●	●	●	●	●	●

Task 7(a)

- You are building a reference counting system for a program that has the following C declaration:

```
struct Tree {  
    int value;  
    struct Tree *left;  
    struct Tree *right;  
};  
typedef struct Tree Tree;
```

- You can enter your answers to the questions below in the file `tree.c`.

Task 7(b)

- As you are compiling a `Tree` program, you are going to have to make modifications to the `Tree` declarations and to any pointer-manipulating statements in the program to make sure that the reference counts are properly updated and that garbage cells are collected.

1. How would you modify the `Tree` declaration?

2. How would you modify this statement:

```
Tree *t = (Tree*)malloc(sizeof(Tree));
```


Task 7(c)

- For the following questions you can assume that `s` and `t` are either `NULL` or point to a previously allocated `Tree` object:

3. How would you modify this statement:

```
t = s;
```

4. How would you modify this statement:

```
t->left = s;
```

5. How would you modify this statement:

```
t->left = s->right;
```

6. How would you modify this statement:

```
t->left = NULL;
```

```
struct Tree {  
    int value;  
    struct Tree *left;  
    struct Tree *right;  
};  
typedef struct Tree Tree;
```




DEFINITIONS AND ALGORITHMS

Reference Counting Algorithm

- Give every object a **reference_count (RC)** field.
- RC holds the number of pointers which point to the object.
- Cases:
 1. **if** (pointer P is made to point to object O) $O.RC++$;
 2. **if** (pointer P no longer points to object O) $O.RC--$;
 3. **if** ($O.RC == 0$) put O on the free list;

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA